Stepwise

Milestone 2 · MLOps Infrastructure

AC215 · Fall 2026 · Report updated 15 October 2026

Team: Bei Li · Ernest Ferdian · Dimitris Pipinis · Xiahui Huang

1. Scope and milestone coverage

Stepwise helps IELTS Academic Writing Task 2 learners revise argumentative essays. Users type or paste a question and draft, receive four criterion scores with specific improvements, then select a target band to generate a model essay with qualitative feedback. Optional position and idea inputs can guide the new version. Task 2 provides a consistent argumentative task for rubric-based analysis.

MS2 delivers a working Google Gemini application, official-source RAG, a reproducible container pipeline and real execution evidence. The text-only project follows the LLM track; image upload and OCR are outside scope.

Project links

GitHub repository: https://github.com/dwcqwcqw/stepwise

Current implementation: https://github.com/dwcqwcqw/stepwise/tree/bei-li

MS2 snapshot branch: https://github.com/dwcqwcqw/stepwise/tree/milestone2

Live frontend: https://stepwise-one.vercel.app

Requirements: https://harvard-iacs.github.io/2026-AC215/milestone2/

Requirement 1 — Virtual environment (Objective + Deliverable)

Implementation: Cloud Shell provides the Linux VM for Docker development; the deployed API runs in a Cloud Run container. docs/setup.md documents prerequisites, authorization, build, startup and troubleshooting.

Evidence: Figure 1 and docs/evidence/cloud-running.png show the running development environment; pipeline.log records healthy API/web/Qdrant and successful data stages. deploy/README.md explains production setup.

Requirement 2 — End-to-end containers (Objective + Deliverables)

Implementation: services/{ingest,chunk,index,api,web} each has a Dockerfile, pyproject.toml and uv.lock. compose.yaml orders ingestion → chunking → indexing → API → web, with Qdrant health checks. After the documented environment/auth setup, make run starts the complete pipeline.

Data versions: ingestion, chunking and index manifests record source SHA-256 hashes, snapshot IDs, embedding versions and record counts. Each downstream stage consumes the preceding manifest, preserving a traceable path from downloaded source to retrieved context.

Evidence: docs/setup.md provides exact commands; pipeline.log and sample artifacts demonstrate the complete container workflow. docs/evidence/band-paragraph-audit.json verifies descriptor extraction, and band-matching-integration.json records query-to-context and model execution checks.

Requirement 3 — LLM RAG (Objective + Deliverables)

Implementation: official band tables are segmented by task, criterion and integer band, then into individual paragraph statements. Chunks retain source hashes, PDF pages, parent IDs and limiting features. Embeddings are stored in Qdrant. Draft scoring retrieves the complete Task 2 band ladder; generation retrieves guidance around the requested level.

Evidence: docs/evidence/retrieval_example.json contains the query, filters and returned source text/page links; rag_verification.json checks all 40 Task 2 criterion-band cases. Section 3 explains chunking, embeddings, database integration and the complete retrieval example.

Requirement 4 — Vision / other modalities

Not applicable to this text-only LLM project. Users type or paste a question and essay; image processing and baseline vision training are outside this milestone's scope.

Requirement 5 — App mock-up and working skeleton (Objective + Deliverable)

Implementation: docs/wireframe.md and Figures 2–5 cover Draft → Feedback → Goals → Model Essay. services/web and services/api implement real UI/backend interaction. Refinements from MS1 include a simpler text editor, criterion-level advice and revision examples, optional generation goals, source-linked feedback and streamed processing status.

Evidence: the frontend supports draft scoring, four-criterion advice, official citations and target-guided essay generation with qualitative commentary. Screenshots and public request receipts are under docs/evidence.

Submission: the course requires deliverables on the milestone2 branch and the full submission commit hash on Canvas by 10:00 PM ET, October 20. Development is on bei-li; the selected release must be copied to milestone2 for submission. The team presentation is 15 minutes plus Q&A, and all members must understand the code and infrastructure.

2. Architecture and cloud environment

Data: ingest → chunk → embed/index → Qdrant. Application: browser → web gateway → FastAPI → retrieved official context → Vertex AI Gemini → validated feedback. Successful data stages and health checks gate startup.

Custom Python services use frozen uv environments; Qdrant supplies the vector database. Compose binds API/web to loopback and does not publish the database port. Google Cloud project stepwise-ac215-20261010 provides Vertex AI and the container backend. Cloud Shell supports development; Vercel and a private Cloud Run API serve the public application.

3. Official knowledge and reproducible RAG

The scoring references and writing guidance are the following two official IELTS documents. Source versions and SHA-256 hashes are recorded in data/sources.lock.json; runtime source files and extracted full text remain outside Git.

- IELTS Writing Band Descriptors (May 2023): all nine pages, both tasks and bands 0–9; 80 parent cells and 238 paragraph statements (119 for Task 2), including the merged Band 0 rule and limiting features.
- IELTS Writing Key Assessment Criteria: definitions and 20 detailed checks—Task Response 5; Coherence and Cohesion 5; Lexical Resource 6; Grammatical Range and Accuracy 4.

Descriptor coverage: 80 criterion-band cells and 238 paragraph statements across both tasks. Task 2 contributes 40 cells and 119 statements: Task Response 32, Coherence and Cohesion 33, Lexical Resource 28, and Grammatical Range and Accuracy 26. Parent-child links retain complete band context; exact text comparisons check that statements remain in the correct task, band and criterion.

Draft scoring retrieves all Task 2 bands 0–9, their paragraph statements, criterion definitions and applicable limiting rules. This complete coverage prevents a small semantic top-k result from omitting relevant score boundaries. Generation uses official descriptors around the selected target, together with the 20 detailed checks. Half-band targets use a combination of published integer-band guidance.

Vectorization: FastEmbed 0.7.3 runs BAAI/bge-small-en-v1.5 as CPU ONNX, producing 384-dimensional vectors. Qdrant uses cosine similarity and metadata filters for task, criterion, band and record type. Source and chunk manifests, model-weight hashes and stable IDs support reproducible indexing. Compose uses a Qdrant server; Cloud Run loads bundled vectors into a replica-local in-memory Qdrant instance.

Retrieval example: POST /api/retrieve with criterion TR, target_band 6.5 and a query about developing examples returns only Task 2 TR descriptors for bands 6 and 7, with text, source/page links and similarity values. The request and complete response are saved in docs/evidence/retrieval_example.json. Similarity selects context; it is not an essay score.

The knowledge base supplies assessment standards and writing guidance. Essay collections and adaptation candidates are excluded from the current RAG workflow; open-model training is outside this implementation's scope.

4. Real model workflow and application

Google Vertex AI gemini-3.5-flash assesses Task Response, Coherence and Cohesion, Lexical Resource, and Grammatical Range and Accuracy in four parallel calls. Each criterion is checked against all ten integer bands and their paragraph statements. The server validates evidence references, binds official citations, applies limiting features and derives the supported band. A failed criterion can receive one targeted repair without repeating successful criteria. Feedback includes a clear next step and a concrete revision example.

Draft assessment does not receive the learner's target band. Four integer criterion scores have equal weight; the server calculates their exact mean and rounds half-up to the nearest 0.5 for the displayed Task 2 practice estimate. For example, (3 + 4 + 3 + 3) / 4 = 3.25 → 3.5. Official descriptors provide

integer-band rows, so the system creates neither half-band descriptor text nor numerical paragraph sub-scores. This estimate covers the submitted Task 2 essay only.

Generation: the learner selects a target band. Position and ideas are blank by default and optional. A blank field preserves the corresponding stance or relevant ideas from the original essay; an explicit new position takes precedence, with supporting ideas adapted as needed.

Gemini writes the model essay, then a single commentary call explains the actual text across four criteria, with practical advice, example refinements and the 20 detailed skill checks. The displayed number is the user's Target band. Generated essays receive qualitative commentary without numerical reassessment or repeated score-matching rewrites.

Progress and privacy: generation shows two steps—Write your essay and Prepare your feedback—with backend progress streamed to the browser. Learner text is sent to Google for inference and is not persisted by Stepwise. Production uses Vercel OIDC and the Cloud Run runtime identity; no Cloud Shell token helper is needed.

5. Verification and exact reproduction

Verification covers complete descriptor extraction, task/criterion/band filtering, exact essay references, official citations, limiting rules and score arithmetic. Generation checks cover optional-field inheritance, explicit stance overrides, qualitative commentary and preservation of the selected target label. Tests also guard against invoking numerical assessment from the generation path. Reproduction logs and sample input/output artifacts are retained under docs/evidence.

Recorded public checks: a target-7.5 request produced a 311-word essay with four commentary panels in 16 seconds. With both optional inputs blank, a 301-word result preserved the source essay's road-priority stance; an explicit public-transport stance produced a 282-word result that followed the override while adapting the original examples. The latter requests took 23 and 17 seconds. These are observed functional runs, not latency guarantees or grading-accuracy measurements.

```
git clone --branch bei-li https://github.com/dwcqwcqw/stepwise.git

cd stepwise

export GOOGLE_CLOUD_PROJECT=stepwise-ac215-20261010

export STEPWISE_AUTH_DIR=/dev/shm/stepwise-auth

python3 scripts/cloud-auth.py > /tmp/stepwise-auth.log 2>&1 &

make run
```

Prerequisites: Docker/Compose, make, Python 3, an authorized gcloud session and an enabled billing/Vertex project. Open localhost:8080 or Cloud Shell Web Preview on port 8080. docs/setup.md contains full configuration and troubleshooting. make verify repeats real model calls; make evidence captures existing outputs/logs; make test runs without Google calls. The Cloud Shell preview is temporary.

6. Public deployment and verification

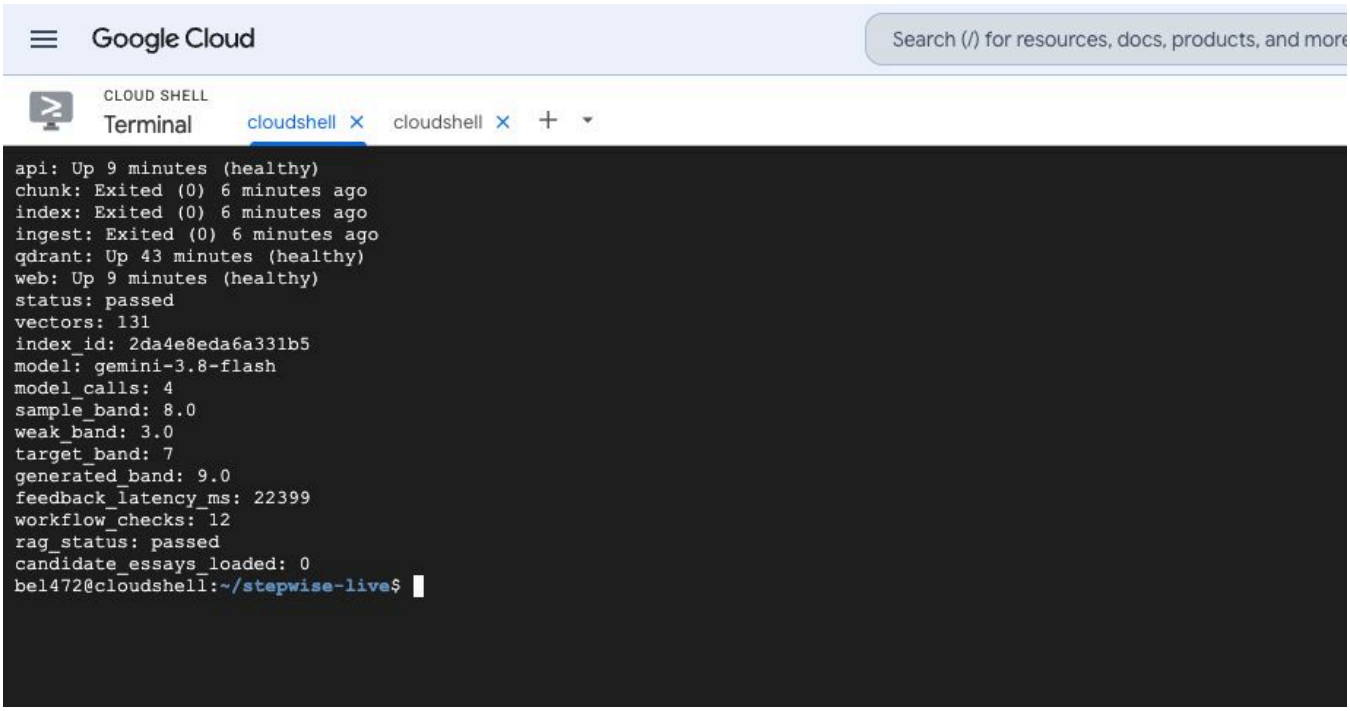
Live application: https://stepwise-one.vercel.app. Implementation: https://github.com/dwcqwcqw/stepwise/tree/bei-li. The recorded application source is cfbedd1144018d1572aa6c5205145c1622685823. docs/evidence/optional-goals-public.json records the

deployment and successful blank-field/override checks; commentary-public.json records all four qualitative panels. The public readiness endpoint was checked on 15 October 2026 and returned ready with gemini-3.5-flash and the optional-goals prompt version.

Cloud connection: Vercel → short-lived OIDC identity → private Cloud Run → Vertex AI Gemini. The API loads a bundled rubric vector index and pinned embedding model. Separate runtime, invoker and build identities control access. Google credentials remain server-side, and the deployed service runs independently of Cloud Shell.

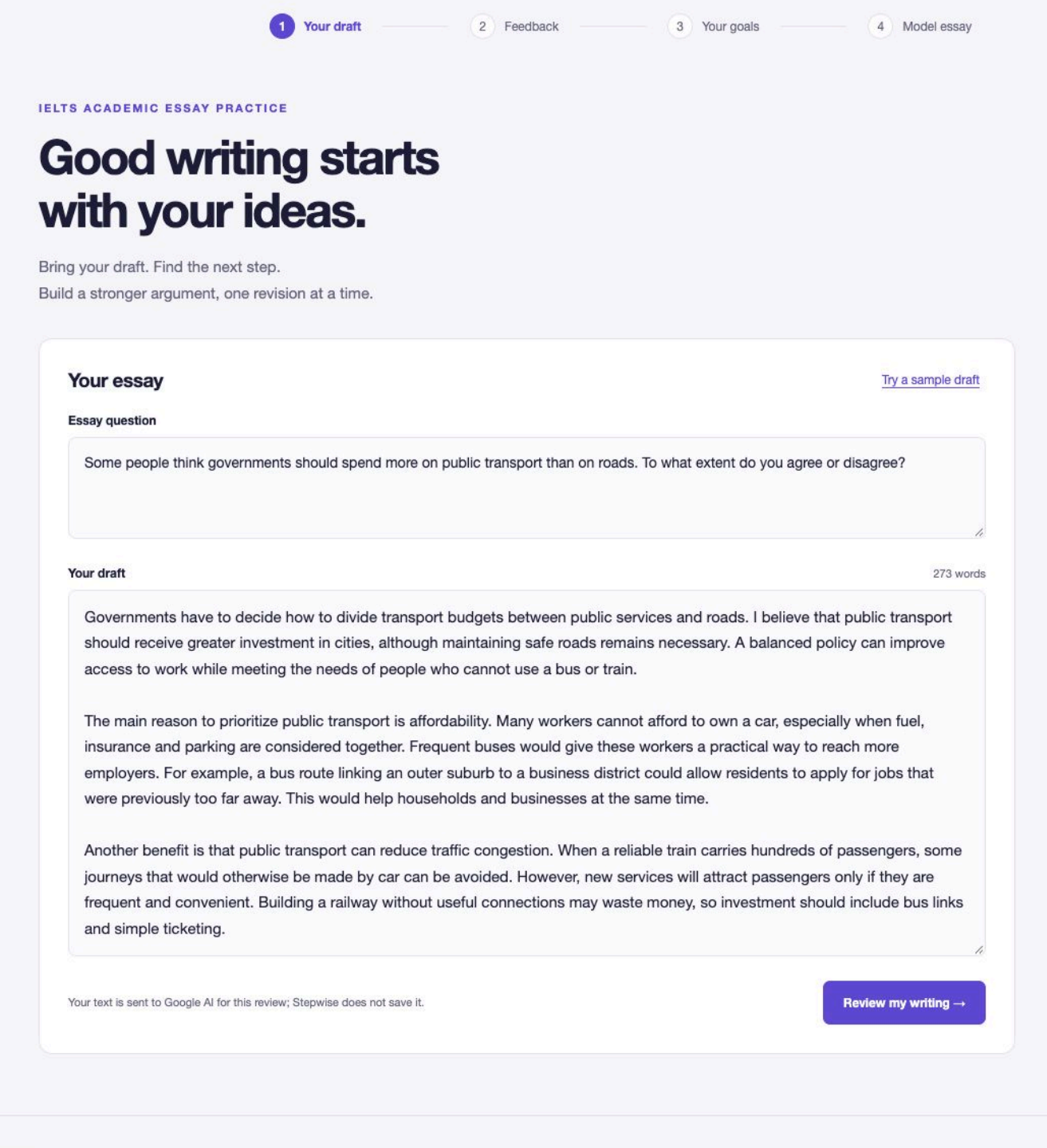
Public verification inspects the target label, returned essay, four commentary tabs, actionable examples and official source links. The generated Target band is the selected writing goal, not a separately assessed score. Draft scoring remains a distinct workflow with four criterion grades and a calculated mean. Source files, execution receipts and screenshots support reproducibility; scoring consistency and broader input coverage remain evaluation priorities.

Figure 1. Running cloud environment



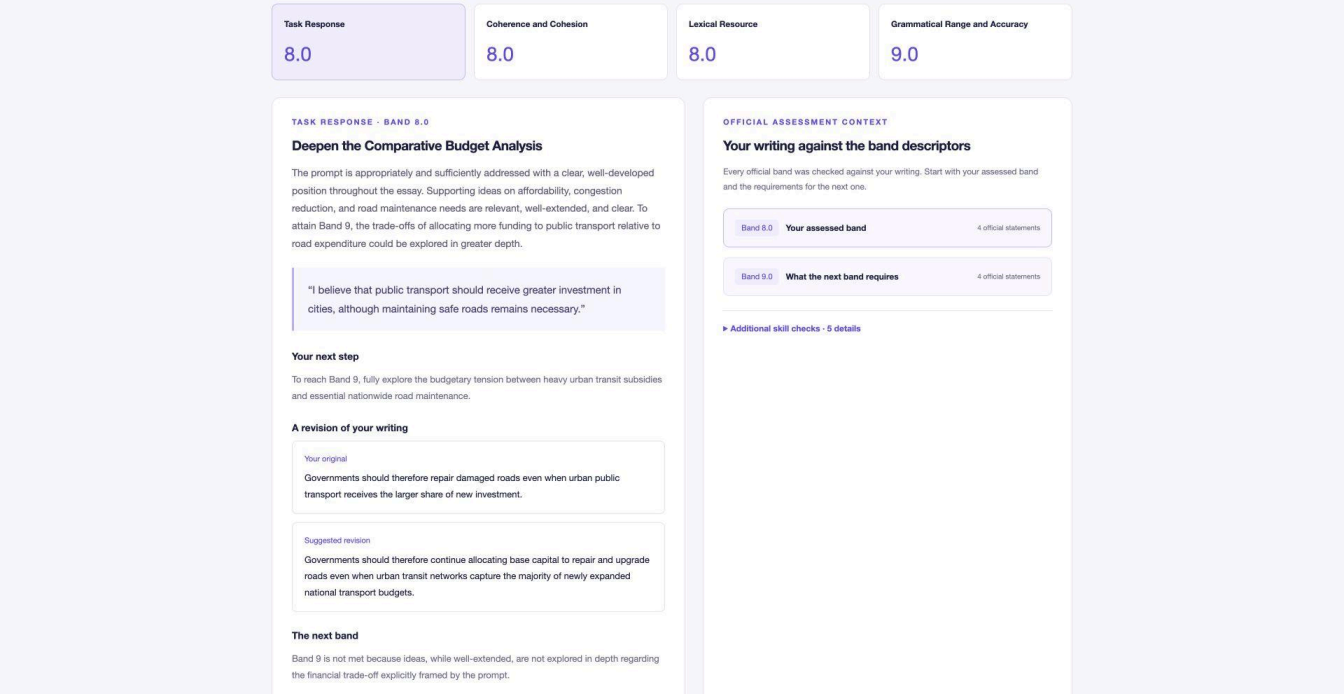
Cloud Shell environment: API, web and Qdrant run in containers; ingest, chunk and index complete successfully. Pipeline logs, manifests and sample input/output artifacts are available in the repository.

Figure 2. Your draft



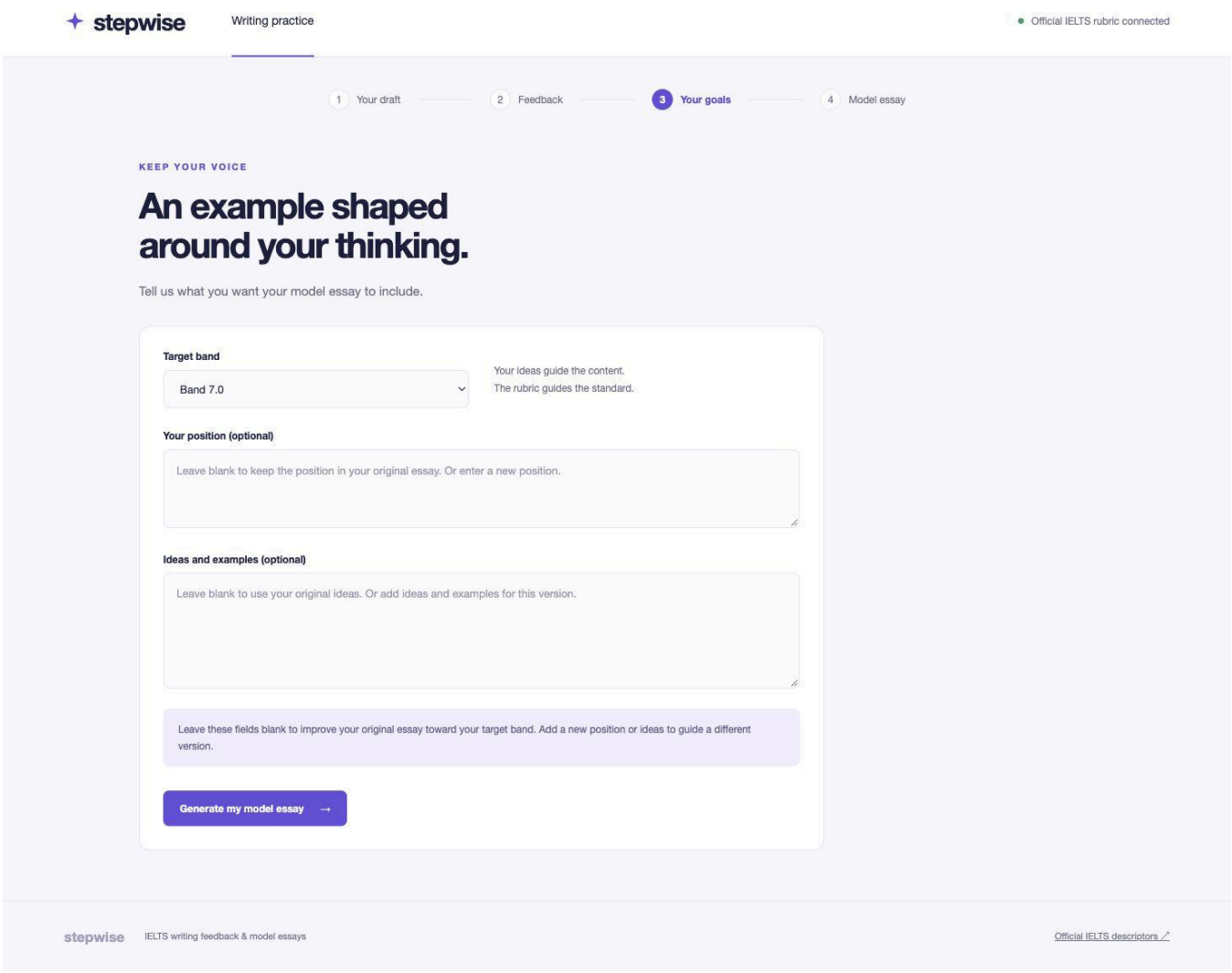
One editor accepts a typed or pasted question and essay, with a sample draft and live word count. The decorative panel, initial target sidebar and image/OCR controls are absent.

Figure 3. Criterion feedback and band matching



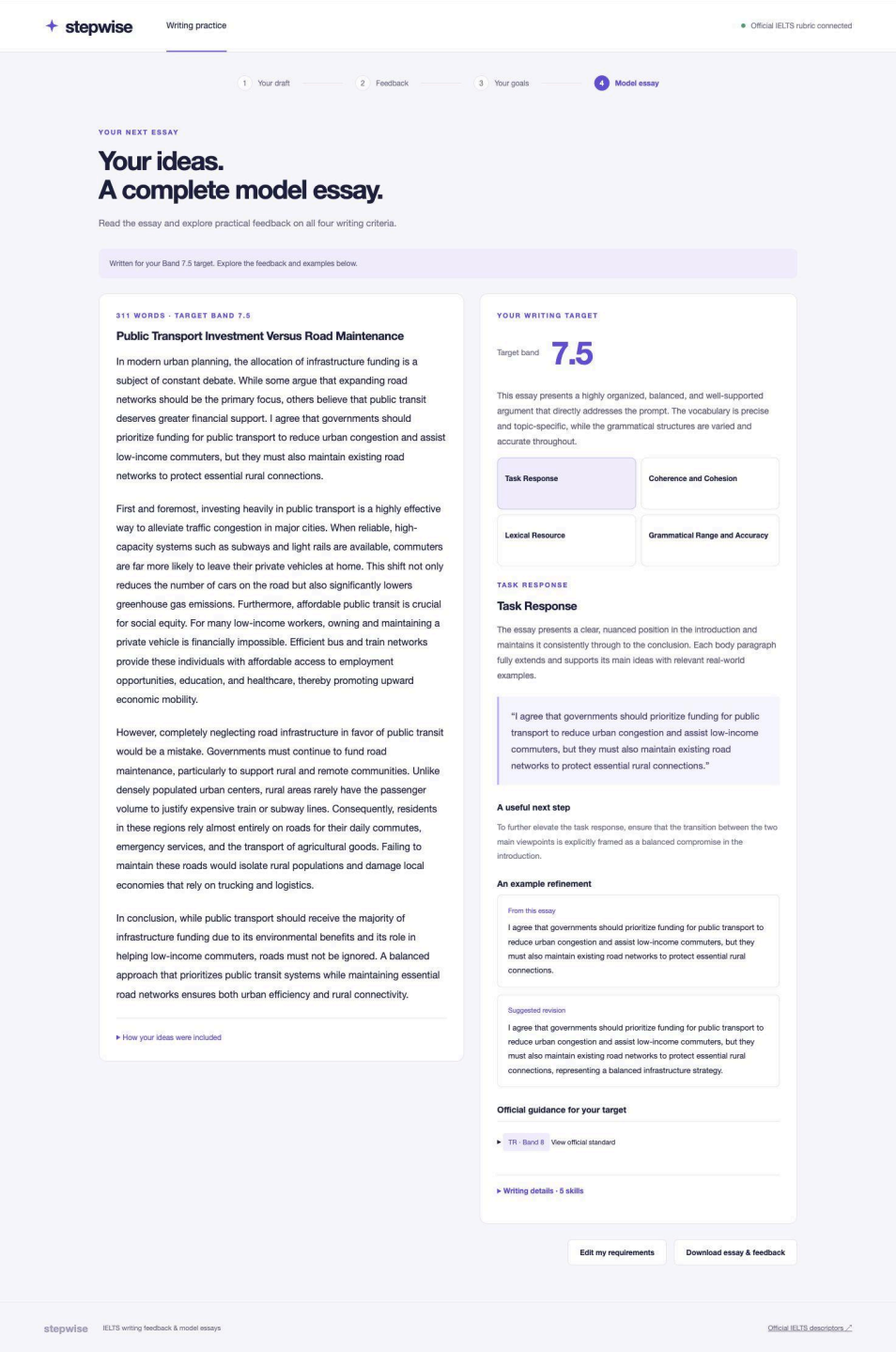
Feedback shows the assessed and next-band official statements, exact essay evidence, revision advice and expandable skill details. Four equally weighted criterion scores, their exact mean and the half-band estimate appear above the feedback.

Figure 4. Your goals



The learner selects a target band. Position and ideas are optional and blank by default; blank inputs preserve the original essay's stance and relevant ideas, while explicit inputs guide the new version.

Figure 5. Generated essay and qualitative feedback



The result shows the complete model essay, the selected Target band, and four qualitative feedback panels with advice, example refinements and official guidance. Generation uses two processing steps and does not numerically reassess the essay.